

Home Assignment
Empirical Industrial Organization
ECONM0013

Student ID: 2676835

April 2026

Contents

1	Introduction	1
2	Estimation and Simulation of Entry Games	2
2.1	Question A – Derivation of Log-Likelihood, Gradient and Hessian	2
2.2	Question B – Maximum Likelihood Estimation	6
2.3	Question C – Results	7
2.4	Question D – STATA Comparison	9
2.5	Question E – Simulation	9
2.6	Question F – Firm Heterogeneity Discussion	12
3	Article Discussion	15
	References	17
A	Appendix	18
A.1	Code	18
A.1.1	README	18
A.1.2	STATA Do-file	18
A.1.3	Question B – Main Script	20
A.1.4	Question B – Binary Probit Function	21
A.1.5	Question C – Standard Errors and Results	24
A.1.6	Question E – Simulation	24

1 Introduction

Entry models are defined as models where the dependent variable is whether a firm decides to enter a market or not. They often have sunk or fixed costs which are often unobservable to the econometrician and any payoffs associated with entering are influenced by the number of other entrants to the market. We may want to estimate structural entry models to estimate these parameters or to control for any endogenous entry or exit.

One of the key challenges in the structural entry model literature is the prevalence of multiple Nash equilibria, and many papers have attempted to address this issue. Bresnahan and Reiss (1991) offer one solution by assuming symmetry of potential entrants and thereby estimating the number of active firms in a market, rather than the identity of said firms. They estimate an ordered probit model where each additional entrant requires a larger market size to make non-negative profits. They find that competitive conduct changes rapidly as the number of incumbent firms rises, but has little effect beyond three to five firms.

Berry (1992) moves away from symmetry when estimating a structural entry model in the airline industry. Instead, Berry (1992) allows firm-level heterogeneity by introducing idiosyncratic shocks, meaning the identity of entering firms is also estimated, rather than solely the number of entrants, as in Bresnahan and Reiss (1991). To address the multiplicity issue, Berry (1992) imposes an order of entry assumption. He does this in two ways: most profitable firms move first and incumbent firms move first. Since the resulting likelihood involves high-dimensional integrals which may be computationally infeasible, Berry (1992) estimates the model using a Simulation.

This assignment estimates a simpler version of the model by Bresnahan and Reiss (1991), using their original data on tire dealers. Then a simplified version of the entry model used by Berry (1992) is also estimated on the same dataset. Finally, the article by Toivanen and Waterson (2005) is discussed.

2 Estimation and Simulation of Entry Games

2.1 Question A – Derivation of Log-Likelihood, Gradient and Hessian

The simplified version of Bresnahan and Reiss (1991) focuses on the potential monopolistic markets of tyre dealers. The following binary entry model is considered:

$$N_m = \begin{cases} 1 & \text{if } \Pi_{1m} = \bar{\Pi}_{1m} + \varepsilon_m > 0 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where Π_{1m} is market m tyre dealer's profit, $\bar{\Pi}_{1m}$ is market m tyre dealer's expected profit, and $\varepsilon_m \sim \mathcal{N}(0, 1)$. $\bar{\Pi}_{1m}$ is further assumed to be:

$$\bar{\Pi}_{1m} = S(\mathbf{Y}, \boldsymbol{\lambda}) \cdot V_1(\mathbf{Z}, \alpha_1, \boldsymbol{\beta}) - F_1(\mathbf{W}, \boldsymbol{\gamma}) \quad (2)$$

where $S(\mathbf{Y}, \boldsymbol{\lambda})$ captures market size through local demographic variables $\mathbf{Y}$; $V_1(\mathbf{Z}, \alpha_1, \boldsymbol{\beta})$ represents per-capita variable profits determined by demand shifters $\mathbf{Z}$; and $F_1(\mathbf{W}, \boldsymbol{\gamma})$ represents the fixed costs faced by a potential entrant, driven by cost shifter $\mathbf{W}$. They are assumed to depend linearly on the regressors:

$$S(\mathbf{Y}, \boldsymbol{\lambda}) = \lambda_0 \text{tpop}_m + \lambda_1 \text{opop}_m + \lambda_2 \text{ngrw}_m + \lambda_3 \text{pgrw}_m + \lambda_4 \text{octy}_m, \quad \lambda_0 = 1 \quad (3)$$

$$V_1(\mathbf{Z}, \alpha_1, \boldsymbol{\beta}) = \alpha_1 + \beta_1 \text{eld}_m + \beta_2 \text{pinc}_m + \beta_3 \text{lnhdd}_m + \beta_4 \text{ffrac}_m \quad (4)$$

$$F_1(\mathbf{W}, \boldsymbol{\gamma}) = \gamma_1 + \gamma_L \text{landv}_m \quad (5)$$

These assumptions imply that

$$\Pr[N_m = 1 \mid \mathbf{Y}, \mathbf{Z}, \mathbf{W}] = \Phi(\bar{\Pi}_{1m}) \quad (6)$$

where $\Phi(\cdot)$ is the Cumulative Distribution Function (CDF) of the standard normal distribution, meaning entry in market m follows a binary probit structure.

I define $\boldsymbol{\theta}$ as a vector of the free parameters introduced in Equations (3), (4) and (5), that

is:

$$\boldsymbol{\theta} = \begin{pmatrix} \boldsymbol{\lambda} \\ \alpha_1 \\ \boldsymbol{\beta} \\ \gamma \end{pmatrix} = \begin{pmatrix} \lambda_1 \\ \lambda_2 \\ \lambda_3 \\ \lambda_4 \\ \alpha_1 \\ \beta_1 \\ \beta_2 \\ \beta_3 \\ \beta_4 \\ \gamma_1 \\ \gamma_L \end{pmatrix} \quad (7)$$

Note that λ_0 is excluded from $\boldsymbol{\theta}$ as it is constrained to equal 1 and is therefore not a free parameter to be estimated.

The log-likelihood function of the binary probit model is defined as:

$$\begin{aligned} \ell(\boldsymbol{\theta}) &= \sum_{m=1}^M (N_m \ln [\Phi(\bar{\Pi}_{1m})] + (1 - N_m) \ln [1 - \Phi(\bar{\Pi}_{1m})]) \\ &= \sum_{m=1}^M \left(N_m \ln [\Phi(S(\mathbf{Y}, \boldsymbol{\lambda})V_1(\mathbf{Z}, \alpha_1, \boldsymbol{\beta}) - F_1(\mathbf{W}, \boldsymbol{\gamma}))] \right. \\ &\quad \left. + (1 - N_m) \ln [1 - \Phi(S(\mathbf{Y}, \boldsymbol{\lambda})V_1(\mathbf{Z}, \alpha_1, \boldsymbol{\beta}) - F_1(\mathbf{W}, \boldsymbol{\gamma}))] \right) \end{aligned} \quad (8)$$

However, since $\bar{\Pi}_{1m}$ is non-linear in the parameters $\boldsymbol{\theta}$, the gradient and hessian become slightly more difficult to derive. Using the chain rule, the gradient is defined as:

$$\frac{\partial \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \sum_{m=1}^M \left(\frac{\partial \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}} \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}} \right) \quad (9)$$

We can obtain the first term in the summation by differentiating Equation (8) with respect to $\bar{\Pi}_{1m}$:

$$\frac{\partial \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}} = \frac{N_m \phi(\bar{\Pi}_{1m})}{\Phi(\bar{\Pi}_{1m})} - \frac{(1 - N_m) \phi(\bar{\Pi}_{1m})}{1 - \Phi(\bar{\Pi}_{1m})} \quad (10)$$

where $\phi(\cdot)$ is the standard normal Probability Density Function (PDF), and $\Phi(\cdot)$ is as defined in Equation (6).

The second term is slightly more complex, since $\bar{\Pi}_{1m}$ is non-linear in $\boldsymbol{\theta}$, meaning each

element of $\frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}}$ must be derived individually. It can be written as:

$$\frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}} = \begin{pmatrix} \frac{\partial \bar{\Pi}_{1m}}{\partial \lambda} \\ \frac{\partial \bar{\Pi}_{1m}}{\partial \alpha_1} \\ \frac{\partial \bar{\Pi}_{1m}}{\partial \beta} \\ \frac{\partial \bar{\Pi}_{1m}}{\partial \gamma} \end{pmatrix} = \begin{pmatrix} \text{opop}_m \cdot V_{1m} \\ \text{ngrw}_m \cdot V_{1m} \\ \text{pgrw}_m \cdot V_{1m} \\ \text{octy}_m \cdot V_{1m} \\ S_m \\ S_m \cdot \text{eld}_m \\ S_m \cdot \text{pinc}_m \\ S_m \cdot \text{lnhdd}_m \\ S_m \cdot \text{frac}_m \\ -1 \\ -\text{landv}_m \end{pmatrix} \quad (11)$$

Combining Equations (9), (10) and (11), the full gradient is:

$$\frac{\partial \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = \sum_{m=1}^M \left(\frac{N_m \phi(\bar{\Pi}_{1m})}{\Phi(\bar{\Pi}_{1m})} - \frac{(1 - N_m) \phi(\bar{\Pi}_{1m})}{1 - \Phi(\bar{\Pi}_{1m})} \right) \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}} \quad (12)$$

where $\frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}}$ is as defined in Equation (11).

The Hessian can be derived by differentiating Equation (12) with respect to $\boldsymbol{\theta}'$ using the product and chain rules:

$$\frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}'} = \sum_{m=1}^M \left(\frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}^2} \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}} \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}'} + \frac{\partial \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}} \cdot \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}'} \right) \quad (13)$$

The terms in Equation (13) are derived in turn. $\frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}^2}$ can be derived by differentiating Equation (10) with respect to $\bar{\Pi}_{1m}$:

$$\begin{aligned} \frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}^2} = & -N_m \left[\frac{\phi'(\bar{\Pi}_{1m}) \Phi(\bar{\Pi}_{1m}) - \phi(\bar{\Pi}_{1m})^2}{\Phi(\bar{\Pi}_{1m})^2} \right] \\ & - (1 - N_m) \left[\frac{\phi'(\bar{\Pi}_{1m}) (1 - \Phi(\bar{\Pi}_{1m})) + \phi(\bar{\Pi}_{1m})^2}{(1 - \Phi(\bar{\Pi}_{1m}))^2} \right] \end{aligned} \quad (14)$$

Using the fact that for the standard normal distribution $\phi'(x) = -x\phi(x)$, this simplifies to:

$$\begin{aligned} \frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}^2} = & -N_m \left[\frac{\phi(\bar{\Pi}_{1m})^2 + \bar{\Pi}_{1m} \phi(\bar{\Pi}_{1m}) \Phi(\bar{\Pi}_{1m})}{\Phi(\bar{\Pi}_{1m})^2} \right] \\ & - (1 - N_m) \left[\frac{\phi(\bar{\Pi}_{1m})^2 - \bar{\Pi}_{1m} \phi(\bar{\Pi}_{1m}) (1 - \Phi(\bar{\Pi}_{1m}))}{(1 - \Phi(\bar{\Pi}_{1m}))^2} \right] \end{aligned} \quad (15)$$

For the other terms in Equation (13), $\frac{\partial \ell(\boldsymbol{\theta})}{\partial \bar{\Pi}_{1m}}$ and $\frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}}$ were already derived in Equations (10) and (11) respectively. $\frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}}$ is an 11×1 column vector and $\frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}'}$ is its transpose, a 1×11 row vector, so their outer product yields an 11×11 matrix. The final term $\frac{\partial^2 \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}'}$ is complex at first glance, however many terms are zero. Since $\bar{\Pi}_{1m} = S \cdot V_1 - F_1$ contains the product of S and V_1 , the second derivatives are non-zero only when one parameter comes from $\boldsymbol{\lambda}$ and the other from α_1 or β , as these interact through $S \cdot V_1$. All other second derivatives are zero since S , V_1 and F_1 are each linear in their own parameters. The non-zero elements of $\frac{\partial^2 \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}'}$ are:

$$\begin{aligned} \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_1 \partial \alpha_1} &= \text{opop} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_1 \partial \beta_1} &= \text{opop} \cdot \text{eld} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_1 \partial \beta_2} &= \text{opop} \cdot \text{pinc} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_1 \partial \beta_3} &= \text{opop} \cdot \text{lnhdd} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_1 \partial \beta_4} &= \text{opop} \cdot \text{ffrac} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_2 \partial \alpha_1} &= \text{ngrw} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_2 \partial \beta_1} &= \text{ngrw} \cdot \text{eld} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_2 \partial \beta_2} &= \text{ngrw} \cdot \text{pinc} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_2 \partial \beta_3} &= \text{ngrw} \cdot \text{lnhdd} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_2 \partial \beta_4} &= \text{ngrw} \cdot \text{ffrac} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_3 \partial \alpha_1} &= \text{pgrw} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_3 \partial \beta_1} &= \text{pgrw} \cdot \text{eld} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_3 \partial \beta_2} &= \text{pgrw} \cdot \text{pinc} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_3 \partial \beta_3} &= \text{pgrw} \cdot \text{lnhdd} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_3 \partial \beta_4} &= \text{pgrw} \cdot \text{ffrac} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_4 \partial \alpha_1} &= \text{octy} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_4 \partial \beta_1} &= \text{octy} \cdot \text{eld} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_4 \partial \beta_2} &= \text{octy} \cdot \text{pinc} \\ \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_4 \partial \beta_3} &= \text{octy} \cdot \text{lnhdd} & \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_4 \partial \beta_4} &= \text{octy} \cdot \text{ffrac} \end{aligned} \quad (16)$$

with all other elements equal to zero. The matrix is symmetric, so $\frac{\partial^2 \bar{\Pi}_{1m}}{\partial \beta_k \partial \lambda_j} = \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_j \partial \beta_k}$ for all j, k , and similarly $\frac{\partial^2 \bar{\Pi}_{1m}}{\partial \alpha_1 \partial \lambda_j} = \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \lambda_j \partial \alpha_1}$.

For computational purposes, these expressions can be rewritten to match the MATLAB

implementation. Defining:

$$B_m = \frac{N_m \phi(\bar{\Pi}_{1m})}{\Phi(\bar{\Pi}_{1m})} - \frac{(1 - N_m) \phi(\bar{\Pi}_{1m})}{1 - \Phi(\bar{\Pi}_{1m})} \quad (17)$$

$$\begin{aligned} A_m = & \frac{N_m \phi(\bar{\Pi}_{1m})}{\Phi(\bar{\Pi}_{1m})} \left(-\frac{\phi(\bar{\Pi}_{1m})}{\Phi(\bar{\Pi}_{1m})} - \bar{\Pi}_{1m} \right) \\ & + \frac{(1 - N_m) (-\phi(\bar{\Pi}_{1m}))}{1 - \Phi(\bar{\Pi}_{1m})} \left(\frac{\phi(\bar{\Pi}_{1m})}{1 - \Phi(\bar{\Pi}_{1m})} - \bar{\Pi}_{1m} \right) \end{aligned} \quad (18)$$

the gradient and hessian can be expressed compactly as:

$$\frac{\partial \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} = - \sum_{m=1}^M B_m \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}} \quad (19)$$

$$\frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}'} = - \sum_{m=1}^M \left(A_m \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}} \cdot \frac{\partial \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta}'} + B_m \cdot \frac{\partial^2 \bar{\Pi}_{1m}}{\partial \boldsymbol{\theta} \partial \boldsymbol{\theta}'} \right) \quad (20)$$

where the negative signs arise because `fminunc` (which will be used in MATLAB in the following question) minimises the negative log-likelihood.

2.2 Question B – Maximum Likelihood Estimation

The 11 free parameters, $\boldsymbol{\lambda}$, α_1 , $\boldsymbol{\beta}$ and $\boldsymbol{\gamma}$, are estimated by Maximum Likelihood Estimation (MLE) using MATLAB, adapting the code from Problem Set 7 to accommodate the non-linear structure of the Bresnahan and Reiss (1991) entry model. λ_0 is constrained to 1 and so is not considered a free parameter to be estimated.

The dataset is provided as a `.dta` file from Problem Set 8, so it is first transformed to a `.csv` file in STATA. The do-file for this is provided in the Appendix.

The MATLAB script starts by clearing the workspace and declaring all the global variables, which are the dependent variable N_m and all the regressors. The dataset is then loaded and filtered to only include observations with `tire` less than or equal to one, as we are focusing on monopolistic markets. The original dataset contains 202 observations and after filtering, 84 observations remain. This is consistent with STATA from Problem Set 8.

The key variables are then extracted from the dataset and the optimisation is initialised at $\boldsymbol{\theta}_0 = \mathbf{0}$, a vector of zeros. While in Problem Set 8, we initialised our STATA implementation by searching over parameter ranges using `ml search`, I could not find a direct equivalent in MATLAB. To ensure numerical stability during optimisation, the predicted probabilities

$\Phi(\bar{\Pi}_{1m})$ are bounded away from 0 and 1 by a small tolerance of 10^{-10} :

$$\Phi(\bar{\Pi}_{1m}) = \min \left(\max \left(\Phi(\bar{\Pi}_{1m}), 10^{-10} \right), 1 - 10^{-10} \right) \quad (21)$$

This prevents the log-likelihood from returning undefined values arising from $\ln(0)$ during the iterative optimisation procedure.

To improve precision and speed, I provide the log-likelihood, gradient and hessian formulae to MATLAB as derived in Question A. This function (`binary_probit()`) starts by unpacking θ and computing $S(\mathbf{Y}, \lambda)$, $V_1(\mathbf{Z}, \alpha_1, \beta)$, $F_1(\mathbf{W}, \gamma)$ and $\bar{\Pi}_{1m}$ as defined in Equations (3), (4), (5) and (2) respectively.

$\Phi(\bar{\Pi}_{1m})$ and $\phi(\bar{\Pi}_{1m})$ are evaluated at the current parameter guess using `normcdf` and `normpdf` respectively. The probabilities are then bounded away from 0 and 1 to ensure numerical stability, as discussed above. The negative log-likelihood is then computed directly from Equation (8). B_m and A_m , as defined in Equations (17) and (18), are computed as $M \times 1$ vectors evaluated at every market simultaneously.

The first derivative vector $\frac{\partial \bar{\Pi}_{1m}}{\partial \theta}$ and second derivative matrix $\frac{\partial^2 \bar{\Pi}_{1m}}{\partial \theta \partial \theta'}$, derived in Equations (11) and (16), are constructed as an $M \times 11$ matrix `dPi` and an $M \times 11 \times 11$ matrix `d2Pi` respectively, with non-zero entries filled in explicitly.

Finally, the gradient and hessian are computed using the compact Equations (19) and (20) using `for` loops over the 11 parameters as in Problem Set 7.

The negative log-likelihood is minimised using `fminunc` with the trust-region algorithm, following the same approach as Problem Set 7. The gradient and hessian derived in Question A are supplied directly to the optimiser via the `GradObj` and `Hessian` options, rather than relying on numerical approximations. Convergence tolerances are set to `TolFun` = `TolX` = 10^{-10} to ensure a precise solution. The estimates are provided in Table 1 and are interpreted and compared to STATA estimates in Questions C and D respectively.

2.3 Question C – Results

Standard errors are computed as follows:

$$\widehat{\text{se}}(\hat{\theta}) = \sqrt{\text{diag}(H^{-1})} \quad (22)$$

where $H = \frac{\partial^2 \ell(\theta)}{\partial \theta \partial \theta'}$ is the hessian as defined in Equation (13), evaluated at $\hat{\theta}$.

Z-statistics and p-values are computed from the asymptotic normality of the MLE. The

z-statistic for each parameter is:

$$z_k = \frac{\hat{\theta}_k}{\widehat{\text{se}}(\hat{\theta}_k)} \quad (23)$$

and p-values are computed as $2(1 - \Phi(|z_k|))$ for a two-sided test.

Table 1: MLE Estimates of the Binary Entry Model

Parameter	Coefficient	Std. Error	Z-statistic	P-value
λ_1 (opop)	7.3978	21.5780	0.3428	0.7317
λ_2 (ngrw)	-12.9663	44.6311	-0.2905	0.7714
λ_3 (pgrw)	21.0392	65.9326	0.3191	0.7496
λ_4 (octy)	-3.3466	15.1399	-0.2210	0.8251
α_1	-0.1630	1.1744	-0.1388	0.8896
β_1 (eld)	0.4308	0.9771	0.4409	0.6593
β_2 (pinc)	0.0306	0.0834	0.3665	0.7140
β_3 (lnhdd)	-0.0072	0.1393	-0.0515	0.9589
β_4 (ffrac)	0.1822	0.3976	0.4582	0.6468
γ_1	0.8025**	0.3660	2.1927	0.0283
γ_L (landv)	-0.7250	0.7273	-0.9968	0.3189

Notes: Significance codes: *** < 0.01, ** < 0.05, * < 0.1.

Table 1 reports the MLE estimates of the 11 structural parameters. All parameters are statistically insignificant, with the exception of the intercept term γ_1 , which is significant at the 5% level ($p = 0.028$). This is consistent with the findings of Bresnahan and Reiss (1991), who find the majority of their specifications contain insignificant demand and cost coefficients in their baseline estimates. They interpret this as evidence that the sample restrictions have provided a homogeneous market where these variables add little explanatory power. In our case, restricting the sample to only include potentially monopolistic markets with zero or one tyre dealer likely produces a similar homogeneous sample.

The significant positive estimate of γ_1 (0.8025) implies fixed costs have a positive intercept, consistent with the existence of meaningful entry costs in the tyre dealer market, such as the acquisition of specialist machinery or licensing. The negative estimate of γ_L (-0.7250) suggests land value is negatively associated with fixed costs, which is surprising as one might expect the opposite to be true as tyre dealers require physical premises to operate. However, this result is statistically insignificant and so must be interpreted with caution.

It is also worth noting that with only 84 observations and 11 parameters to estimate, the model is rather parameter-heavy, likely contributing to the relatively large standard errors. Bresnahan and Reiss (1991) acknowledge a similar issue in their own baseline estimates.

2.4 Question D – STATA Comparison

Table 2 reports the MLE estimates of the 11 structural parameters from Problem Set 8. Comparing Tables 1 and 2, we can see that the coefficients are almost identical across the two implementations, differing only by around 0.01 or less. The standard errors, z-statistics and p-values are also very similar across the two implementations. The minor differences could be due to a variety of reasons; for example, they could be due to numerical precision as the two software packages handle float values slightly differently, or due to the different optimisation algorithms - as MATLAB uses trust-region while STATA defaults to using Newton-Raphson. The two also have slightly different convergence criteria which may also result in marginally different final parameter values.

Table 2: STATA MLE Estimates of the Binary Entry Model

Parameter	Coefficient	Std. Error	Z-statistic	P-value
λ_1 (opop)	7.4033	21.4677	0.34	0.730
λ_2 (ngrw)	-12.9781	44.3233	-0.29	0.770
λ_3 (pgrw)	21.0576	65.4582	0.32	0.748
λ_4 (octy)	-3.3490	15.1312	-0.22	0.825
λ_0 (tpop)	1 (constrained)			
α_1	-0.1631	1.1730	-0.14	0.889
β_1 (eld)	0.4307	0.9752	0.44	0.659
β_2 (pinc)	0.0306	0.0828	0.37	0.712
β_3 (lnhdd)	-0.0071	0.1388	-0.05	0.959
β_4 (ffrac)	0.1821	0.3945	0.46	0.644
γ_1	0.8025**	0.3658	2.19	0.028
γ_L (landv)	-0.7250	0.7273	-1.00	0.319

Notes: λ_0 is constrained to equal 1 and is therefore not reported.
Significance codes: *** < 0.01, ** < 0.05, * < 0.1.

Overall, the similarity between Tables 1 and 2 validates the MATLAB code and provides strong evidence that both optimisation algorithms have converged to the same stationary point of the likelihood function.

2.5 Question E – Simulation

This question introduces firm heterogeneity into the model and estimates the expected number of entrants for each market using a simulation, following Berry (1992). The STATA estimates from the full model of Question B in Problem Set 8 are exported (see do-file in Appendix) and used for the simulation. The profit function is adapted to have firm

heterogeneity in each market:

$$\Pi_{im}(n) = S_m \cdot V_m(n) - F_m(n) + \varepsilon_{im} \quad (24)$$

where ε_{im} is an error term specific to firm i in market m which is known to the firm but unobserved for the econometrician, and n is the number of firms in market m . The ε_{im} 's are assumed to be i.i.d. across firms and markets, and follow the standard normal distribution, i.e. $\varepsilon_{im} \sim \mathcal{N}(0, 1)$. The entry condition is defined as:

$$\Pi_{im}(n) \geq 0 \iff \varepsilon_{im} \geq -(S_m \cdot V_m(n) - F_m(n)) \quad (25)$$

That is, firm i will enter market m if and only if its firm-specific shock ε_{im} is sufficiently large to make entry profitable, given n firms in the market.

The MATLAB code starts by re-loading the full dataset and loading the estimated coefficients from STATA. The dataset is filtered to remove markets with more than four firms and the number of potential entrants in each market (N) is set to four. Following Bresnahan and Reiss (1991), variable profits per firm decline with entry as competition increases, and is assumed to have the linear form:

$$V_m(n) = \alpha_1 + \mathbf{Z}\boldsymbol{\beta} - \sum_{k=2}^n \alpha_k, \quad n = 1, 2, 3, 4 \quad (26)$$

where a positive α_k indicates lower variable profits when more firms are in the market. Similarly, fixed costs are assumed to be higher for later entrants. For example, the cheapest premises might have already been taken by the earlier entrants. Fixed costs are also assumed to be linear:

$$F_m(n) = \gamma_1 + \gamma_L W_L + \sum_{k=2}^n \gamma_k, \quad n = 1, 2, 3, 4 \quad (27)$$

where a positive γ_k indicates higher costs for later entrants. Market size for each market m is defined the same as Equation (3) from Question A. The expected profits, $\bar{\Pi}_m(n)$, are calculated for each n to give an $M \times N$ matrix of expected profits for each market given n firms in the market, where:

$$\bar{\Pi}_m(n) = S_m \cdot V_m(n) - F_m(n), \quad n = 1, 2, 3, 4 \quad (28)$$

The random seed is then set to one for replicability and the number of simulation draws is set to $S_{\text{draws}} = 1000$ as per the brief. The $M \times 1$ vector $\mathbb{E} \left[n_m^* \mid \mathbf{X}_m, \hat{\boldsymbol{\theta}} \right]$ is initialised to store the expected entrants for each market m , given $\mathbf{X}_m$ (the vector of observables for market m) and $\hat{\boldsymbol{\theta}}$ (the coefficient estimates from STATA). The code then loops over each

market m and for each draw generates $(\varepsilon_{1m}^s, \varepsilon_{2m}^s, \varepsilon_{3m}^s, \varepsilon_{4m}^s)$, a vector of four independent standard normal variables, giving a 1000×4 matrix of $\mathcal{N}(0, 1)$ draws for each market. Within the loop, a 1000×1 vector of n^* 's is initialised and then for each simulation draw s , the following algorithm is implemented using a nested for loop:

1. Start with $n^* = 0$ firms in the market.
2. Set $n = 1$ and calculate $n_{\text{willing}} = \sum_{i=1}^4 1\{\varepsilon_{im}^s \geq -\bar{\Pi}_m(1)\}$. If $n_{\text{willing}} \geq 1$, set $n^* = 1$ and continue to $n = 2$. Otherwise keep $n^* = 0$ and break.
3. Set $n = 2$ and calculate $n_{\text{willing}} = \sum_{i=1}^4 1\{\varepsilon_{im}^s \geq -\bar{\Pi}_m(2)\}$. If $n_{\text{willing}} \geq 2$, set $n^* = 2$ and continue to $n = 3$. Otherwise keep $n^* = 1$ and break.
4. Set $n = 3$ and calculate $n_{\text{willing}} = \sum_{i=1}^4 1\{\varepsilon_{im}^s \geq -\bar{\Pi}_m(3)\}$. If $n_{\text{willing}} \geq 3$, set $n^* = 3$ and continue to $n = 4$. Otherwise keep $n^* = 2$ and break.
5. Set $n = 4$ and calculate $n_{\text{willing}} = \sum_{i=1}^4 1\{\varepsilon_{im}^s \geq -\bar{\Pi}_m(4)\}$. If $n_{\text{willing}} \geq 4$, set $n^* = 4$. Otherwise keep $n^* = 3$.

Finally, n_m^{*s} is averaged over all $S_{\text{draws}} = 1000$ draws to approximate $\mathbb{E}[n_m^* | \mathbf{X}_m, \hat{\boldsymbol{\theta}}]$ for market m :

$$\mathbb{E}[n_m^* | \mathbf{X}_m, \hat{\boldsymbol{\theta}}] \approx \frac{1}{S} \sum_{s=1}^S n_m^{*s} \quad (29)$$

Figure 1 summarises the simulation results and plots the observed number of entrants against the average simulated number of entrants across markets. The observed values only take integers which is the reason why the data points cluster in horizontal bands. There is a clear positive relationship between observed and simulated entrants, providing some evidence that the model is able to capture some of the key determinants of entry. However, most points lie below the 45 degree line, particularly in markets where the observed number of entrants is less than two, indicating a systematic over-prediction. This could be due to a variety of reasons; for example, the model assumes exactly $N = 4$ for all markets but in reality some markets may have fewer. Another reason could be that the assumption that $\varepsilon_{im} \sim \mathcal{N}(0, 1)$ is too restrictive. Finally, it could also be the case that our estimates, $\hat{\boldsymbol{\theta}}$, from Problem Set 8 may not translate well to the Berry (1992) simulation framework.

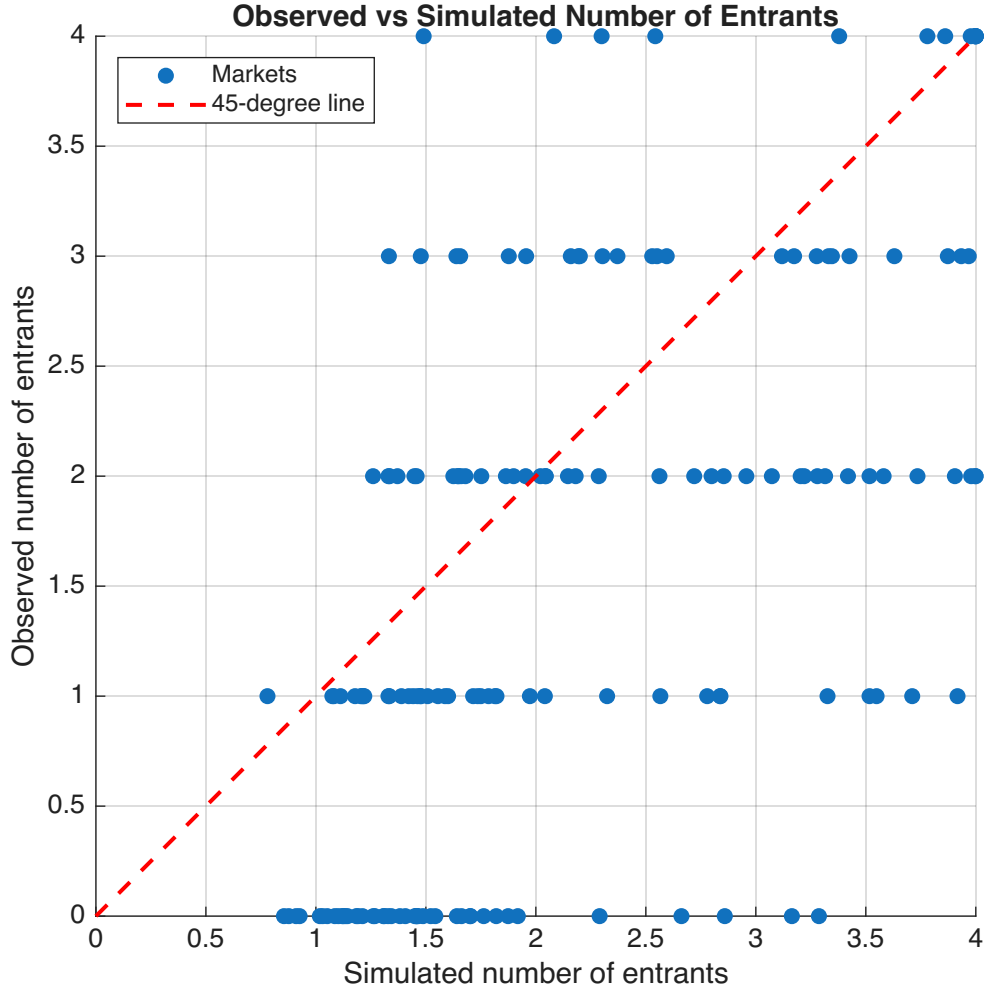


Figure 1: Observed vs Simulated Number of Entrants

2.6 Question F – Firm Heterogeneity Discussion

As mentioned in the Introduction, Bresnahan and Reiss (1991) assume all firms in a given market share the same unobserved profits, that is $\varepsilon_{im} = \varepsilon_m$. Berry (1992) on the other hand, relaxes this assumption and allows firm-level heterogeneity by introducing idiosyncratic shocks, that is $\varepsilon_{im} \neq \varepsilon_m$. Both of these methodologies come with their own advantages and disadvantages which are discussed here.

One of the key advantages of the Bresnahan and Reiss (1991) symmetry assumption is simplicity. The shared error term allows them to estimate entry thresholds using an ordered probit model, with the number of active firms as the dependent variable, which can easily be estimated by MLE. Bresnahan and Reiss (1991) are therefore able to analyse five different industries across 202 isolated towns, each considered to be its own market, using the exact same methodology. This makes their results comparable and interpretable.

Since the assumption requires you to estimate the number of firms in the market rather than the identity of the firms, the case for symmetry depends heavily on the research question. For example, Bresnahan and Reiss (1991) want to analyse the relationship between the number of firms, market size and competition. Thus, there is no need to estimate the identity of firms entering a specific market, rather simply the number of firms. Assuming symmetry in this case is sufficient and their conclusions are clear and informative without identifying the specific firms.

In certain industries, as Bresnahan and Reiss (1991) mention, the symmetry assumption is entirely defensible when firms are genuinely similar. They argue that the five industries they analyse in isolated Midwestern towns use comparable technologies and receive comparable training. Even in the context of tyre dealers in less isolated areas such as Bristol, you might get the odd chain such as a Halfords or Kwik Fit which might benefit from economies of scale, but many tyre dealers are independent firms. And regardless of them being larger tyre dealers, all these firms likely use very similar equipment and the mechanics working at them have all likely received similar training.

An obvious counter-argument to this however, is that in many industries firms differ significantly. Firms can vary in management quality, technology, costs and many other unobservable ways, all of which would violate the assumption that $\varepsilon_{im} = \varepsilon_m$. Berry (1992) highlights that in the airline industry specifically, airport presence (which is partly unobserved to the econometrician) plays a major role in determining profit and can vary across firms. For example, British Airways has a huge presence at London Heathrow, with over 115 destinations from Terminal 5 alone (British Airways, 2026). This means they have a significant share of passenger traffic there, control many gates and likely have established relationships with the airport administration. Compare that with Loganair, who only fly from Heathrow to between three and five destinations across the UK and Ireland (Loganair, 2026). To assume that these two airlines have the exact same unobservable profit determinants would be a very restrictive assumption and could lead to significant bias in all the coefficient estimates. Berry (1992) also finds that firms already operating in both airports of a city-pair combination have a much higher entry rate (6.46%) compared to serving just one city (0.36%) or neither (0.01%).

Table II from Berry (1992) further supports this. There is a strong positive correlation (0.695) between the percentage of markets entered and the percentage of markets exited. Berry highlights that it implies different markets are heterogeneously suited to different firms.

Furthermore, Table VIII from Berry (1992) highlights that ignoring heterogeneity can have significant prediction and policy implications. The ordered probit model (with homogeneity) over-predicts the mean equilibrium number of firms across all markets (1.887) compared

to the prediction of the full model with heterogeneity (1.699), which is much closer to the true value (1.629). This over-prediction is consistent with my answer to Question E, which systematically over-predicts the number of entrants, particularly for lower values of actual entrants. Berry also concludes that the models incorporating firm heterogeneity capture policy-relevant effects better than the simple ordered probit model.

However, despite being more realistic, introducing heterogeneity can introduce other issues. Allowing multiple heterogeneous potential entrants results in having to solve multiple integrals which can be computationally infeasible, even if they are fairly simple integrals themselves (Berry, 1992). As demonstrated in Question E, a solution to this as suggested by Berry (1992) is to conduct a simulation. However, simulations introduce their own assumptions and any finite-sample properties might be harder to assess. For example, an assumption must be made on the distribution of ε_{im} , which we assume follows the standard normal distribution but this can be a rather restrictive assumption as mentioned previously. Furthermore, to overcome the multiplicity problem and be able to identify the entrants, an assumption must be made on the order of entry. Berry does this in two ways: most profitable moves first and incumbents move first. The two specifications gave notably different coefficients, demonstrating the sensitivity to this assumption.

Overall, both assumptions have their own advantages and disadvantages. The "correct" assumption depends heavily on the research question and the industry analysed. While firm heterogeneity may be more realistic in the majority of industries, it comes with its own caveats and other assumptions. Both papers justify why they make their respective assumptions, and they highlight the implications that these assumptions may have on their results.

3 Article Discussion

Standard Industrial Organisation theory suggests that the presence of rival firms should deter entry. However, Toivanen and Waterson (2005) find that this may not be the case using data from the UK counter-service burger multiplant duopoly spanning 1991-1995, a period of mass expansion for the two fast food giants McDonald's and Burger King. They find that rival presence actually increases the probability of entry, attributing this to "learning" about market size. That is, when a firm sees a rival in a particular market, they raise their expectations of the profitability of said market.

One of the fundamental problems though is establishing whether this is genuinely due to learning or simply market attractiveness. For example, going back to airports which often have multiple fast food chains, McDonald's and Burger King would probably independently recognise an airport as a profitable market, so we would likely observe their co-presence (as we often do) without any actual learning having taken place. Toivanen and Waterson (2005) use both a reduced-form and structural estimation to establish this distinction. They exploit timing so that only outlets which were present before a rival entered can have informed it, meaning only lagged rival outlets identify the learning effect while same-period outlets capture the strategic effect.

In relation to the existing literature, this paper builds on the two papers already discussed in depth in earlier sections (Bresnahan and Reiss (1991) and Berry (1992)). Toivanen and Waterson (2005) point out that both of these papers are only able to identify the "average" strategic effect, whereas they allow for between- and within-firm heterogeneity. They use a similar technique to Davis (1999) who also studies multiplant firms' entry decisions, but assumes homogeneous goods. This assumption ignores inter-firm product differentiation, which Toivanen and Waterson (2005) find evidence of and so symmetrical profit functions would lead to biased estimates. This paper builds on all of these papers mentioned and uniquely considers an environment where the same two firms can appear in all of the markets, allowing them to estimate firm-specific profit functions and learning effects.

This paper's most important methodological contribution was being the first paper to estimate firm-specific structural entry equations in a multiplant duopoly environment, which allowed the authors to separate the learning effect from the strategic effect. Combining firm heterogeneity and learning in one framework was completely novel at the time. Alongside the methodological contributions of the paper, it also contributed empirically as they found that the learning effect dominates any competitive effect and that the median distance between a follower's first outlet and the leader's nearest outlet is 253 metres, providing empirical evidence that these firms actively choose to open outlets close to rivals. This is an insightful finding, as it could provide answers as to why rival outlets in fast

food and other retail industries are often found extremely close together; for example, in the coffee shop industry, Starbucks and Costa are also often found in close proximity to each other. Since then, Yang (2012) found consistent evidence of positive spillovers using the same dataset but estimated entry games under incomplete information. Despite this methodological difference, Yang's finding further validates Toivanen and Waterson's results.

I believe the key strength of this paper lies in the unique, clean institutional setting that Toivanen and Waterson have chosen. They justify why the market is a genuine duopoly and why Wimpy (the third largest burger chain) is considered to produce a different good. This specific environment is so unusually clean as it is a genuine duopoly, both firms have centralised decision-making and market exit is a rarity, meaning the same two firms appear across each market consistently. This allows the authors to isolate the effects that actually matter. The fact that decisions are centralised and market exit is rare, means that new outlet locations have meaning rather than being random. Therefore, the finding that followers locate just 253 metres away from leaders is actually a meaningful and interpretable result, rather than a random statistic.

However, this strength can also be seen as a slight limitation, in that it is such a unique environment that it may not generalise well to other industries, especially ones with more players and exits or less centralised decision-making. As mentioned previously, the coffee shop industry with Starbucks vs Costa may provide a similar environment but the majority of retail industries likely would not. A key limitation of the model is that firms are assumed to be myopic, meaning they don't consider future entry decisions when making present ones. Yang (2012) explicitly flags this and points out that a fully dynamic model is required to fully verify the learning effect. This does mean that while Toivanen and Waterson's key finding may be true, it remains unverified. Yang also argues that due to the lack of exit, positive signals are only ever observed, meaning learning remains unverified. Finally, another key limitation is that the identification of the learning effect relies partially on the functional form assumptions, not just timing variation. The authors themselves mention that this approach on its own *"is usually viewed with suspicion"* and that their approach of combining the two can *"also be criticized"* (Toivanen and Waterson, 2005). This does mean unfortunately that the learning effect is not as well separated as the authors suggest.

Overall, despite the limitations, I believe Toivanen and Waterson (2005) have succeeded in answering their question. They answer a question which standard IO theory cannot and I believe they have picked a suitable, albeit rare, institutional setting to show this. The paper does have its critiques and so the results must be taken with a hint of caution.

References

- Berry, S. T. (1992). Estimation of a model of entry in the airline industry. *Econometrica*, 60(4):889–917.
- Bresnahan, T. F. and Reiss, P. C. (1991). Entry and competition in concentrated markets. *Journal of Political Economy*, 99(5):977–1009.
- British Airways (2026). Flights from london heathrow. Available at: <https://www.britishairways.com/content/flights/from-london-heathrow> [Accessed: April 2026].
- Davis, P. J. (1999). Estimation of Cournot games in the presence of indivisibilities and heterogeneous firms. Mimeo, Sloan School of Management, MIT.
- Loganair (2026). Fly london heathrow. Available at: <https://www.loganair.co.uk/flylondonheathrow/> [Accessed: April 2026].
- Toivanen, O. and Waterson, M. (2005). Market structure and entry: Where’s the beef? *The RAND Journal of Economics*, 36(3):680–699.
- Yang, N. (2012). Burger King and McDonald’s: Where’s the spillover? *International Journal of the Economics of Business*, 19(2):255–281.

A Appendix

A.1 Code

A.1.1 README

Input data: BRdata.dta, BRdata.csv

Mapping:

- The STATA do-file converts BRdata.dta to BRdata.csv, estimates the binary probit (Question A of Problem Set 8) and the full ordered probit (Question B of Problem Set 8), and exports the estimates to BR_estimates.csv. The estimates of the full ordered probit model are used in Table 2.
- Questions B produces coefficients in Table 1 while Question C produces Standard Errors, Z-statistic and P-value columns.
- Question E produces Figure 1.

A.1.2 STATA Do-file

```
1 *****
2 ***** Home Assignment *****
3 *****
4 * Clear the workspace
5 clear
6 set more off
7
8 * Import the .dta file
9 use "BRdata.dta", clear
10
11 * Export as a .csv to use for the home assignment
12 export delimited using "BRdata.csv", replace
13
14 *****
15 ***** (Problem Set 8) *****
16 *****
17
18 * The following code comes from the Problem Set 8 solutions on
    Blackboard
```

```

19
20 *****
21 ***** Question A: SINGLE FIRM ENTRY ESTIMATION
22 *****
23
24 drop if tire>1
25 cap program drop monentry
26 program monentry
27 version 1.0
28 args lnf s v f
29 quietly replace `lnf'=ln(normal(`s'*`v'-`f')) if $ML_y1==1
30 quietly replace `lnf'=ln(1-normal(`s'*`v'-`f')) if $ML_y1==0
31 end
32 ml model lf monentry (lambda: tire=opop ngrw pgrw octy, nocons ///
33     offset(tpop)) (beta: tire=eld pinc lnhdd ffrac) (gamma: tire=
34         landv)
35 ml search lambda -1 1 beta 0 1 gammaL -1 1
36 ml max
37
38 *****
39 ***** Question B: MULTIPLE FIRM ENTRY ESTIMATION
40 *****
41
42 use "BRdata.dta", clear
43 cap program drop firmentry
44 program firmentry
45 version 1.0
46 args lnf s v f alpha2 alpha3 alpha4 alpha5 gamma2 gamma3 gamma4
47     gamma5
48 tempvar p2 p3 p4 p5
49 qui gen double `p2'=normal(`s'*(`v'-`alpha2')-`f'-`gamma2')
50     `f'-`gamma2'-`gamma3'-`gamma4')
51 qui gen double `p3'=normal(`s'*(`v'-`alpha2'-`alpha3')-`f'-`gamma2
52     '-`gamma3')
53 qui gen double `p4'=normal(`s'*(`v'-`alpha2'-`alpha3'-`alpha4')
54     ///
55     -`f'-`gamma2'-`gamma3'-`gamma4')
56 qui gen double `p5'=normal(`s'*(`v'-`alpha2'-`alpha3'-`alpha4'-`
57     alpha5') ///
58     -`f'-`gamma2'-`gamma3'-`gamma4'-`gamma5')
59 quietly replace `lnf'=ln(1-normal(`s'*(`v')-`f')) if $ML_y1==0
60 quietly replace `lnf'=ln(normal(`s'*(`v')-`f')-`p2') if $ML_y1==1
61 quietly replace `lnf'=ln(`p2'-`p3') if $ML_y1==2

```

```

56 quietly replace 'lnf'=ln('p3'-'p4') if $ML_y1==3
57 quietly replace 'lnf'=ln('p4'-'p5') if $ML_y1==4
58 quietly replace 'lnf'=ln('p5') if $ML_y1>=5
59 end
60 constraint 1 [alpha4]_cons=0
61 ml model lf firmentry (lambda: tire=opop ngrw pgrw octy, nocons
    ///
62     offset(tpop)) (beta: tire=eld pinc lnhdh ffrac) ///
63     (gamma: tire=landv) (alpha2: tire=) (alpha3: tire=) ///
64     (alpha4: tire=) (alpha5: tire=) (gamma2: tire=) ///
65     (gamma3: tire=) (gamma4: tire=) (gamma5: tire=), constraint(1)
66 ml search lambda 0 50 beta 0 1 alpha2 0 1 alpha3 0 1 alpha4 0 1
    ///
67     alpha5 0 1 gamma2 0 1 gamma3 0 1 gamma4 0 1 gamma5 0 1 gammaL
        -1 1
68 ml max
69
70 *****
71 ***** Home Assignment *****
72 *****
73 * Store coefficients as a matrix and export to CSV for home
    assignment
74 matrix b = e(b)'
75 svmat b, names(coef)
76 export delimited coef1 using "BR_estimates.csv", replace

```

A.1.3 Question B – Main Script

```

1 % Clear workspace and declare global variables
2 clear all;
3 global N tpop opop ngrw pgrw octy eld pinc lnhdh ffrac landv
4
5 % Load and prepare data
6 data = readtable('BRdata.csv');
7
8 % Filter the data and display number of observations
9 disp('Number of observations before filtering:')
10 disp(height(data))
11 data = data(data.tire <= 1, :);
12 disp('Number of observations after filtering:')
13 disp(height(data))

```

```

14
15 % Extract key variables
16 N      = data.tire;
17 tpop   = data.tpop;
18 opop   = data.opop;
19 ngrw   = data.ngrw;
20 pgrw   = data.pgrw;
21 octy   = data.octy;
22 eld    = data.eld;
23 pinc   = data.pinc;
24 lnhdhdd = data.lnhdd;
25 ffrac  = data.ffrac;
26 landv  = data.landv;
27
28 % Start with all coefficients at zero
29 theta_start = zeros(11,1);
30
31 % Estimate Probit by MLE similar to PS7
32 opts = optimset('Algorithm','trust-region','display','iter-
    detailed',...
33     'Diagnostics','on','TolFun',1e-10,'TolX',1e-10,...
34     'GradObj','on','DerivativeCheck','off','Hessian','on');
35
36 % Minimise negative log likelihood
37 [thetahat,~,~,~,~,hessian] = fminunc(@binary_probit, theta_start,
    opts);

```

A.1.4 Question B – Binary Probit Function

```

1 function [llike,grad,hess] = binary_probit(theta)
2 global N tpop opop ngrw pgrw octy eld pinc lnhdhdd ffrac landv
3
4 % Unpack theta
5 lambda = theta(1:4);
6 alpha1 = theta(5);
7 beta   = theta(6:9);
8 gamma  = theta(10:11);
9
10 % Compute S, V1, F1, Pi1
11 S = tpop + lambda(1).*opop + lambda(2).*ngrw + lambda(3).*pgrw +
    lambda(4).*octy; % lambda_0 constrained to 1

```

```

12 V1 = alpha1 + beta(1).*eld + beta(2).*pinc + beta(3).*lnhdd + beta
    (4).*ffrac;
13 F1 = gamma(1) + gamma(2).*landv;
14 Pi1 = S.*V1 - F1;
15
16 % Define Pr as in Equation 6 and negative of log-likelihood
17 M = length(N);
18 param = length(theta);          % 11 parameters to estimate
19 Pr     = normcdf(Pi1);           % Standard normal cdf
20 Pr     = max(Pr, 1e-10);         % prevent log(0)
21 Pr     = min(Pr, 1-1e-10);       % prevent log(0)
22 pdf    = normpdf(Pi1);           % Standard normal pdf
23 llike  = -sum(N.*log(Pr)+(1-N).*log(1-Pr)); % Negative log-
    likelihood function
24
25 % Define B and A as in Equations 17 and 18
26 B      = (N./Pr).*pdf + ((1-N)./(1-Pr)).*(-pdf);
27 A      = (N./Pr).*pdf.*(-pdf./Pr-Pi1) + ((1-N)./(1-Pr)).*(-pdf).*(
    pdf./(1-Pr)-Pi1);
28
29 % Build dPi (M x 11)
30 dPi = zeros(M,11);              % Start with Mx11 matrix of 0s
31 dPi(:,1) = opop .* V1;          % dPi / dlambda1
32 dPi(:,2) = ngrw .* V1;          % dPi / dlambda2
33 dPi(:,3) = pgrw .* V1;          % dPi / dlambda3
34 dPi(:,4) = octy .* V1;          % dPi / dlambda4
35 dPi(:,5) = S;                   % dPi / dalpha1
36 dPi(:,6) = S .* eld;            % dPi / dbeta1
37 dPi(:,7) = S .* pinc;           % dPi / dbeta2
38 dPi(:,8) = S .* lnhdd;          % dPi / dbeta3
39 dPi(:,9) = S .* ffrac;          % dPi / dbeta4
40 dPi(:,10) = -ones(M,1);         % dPi / dgamma1
41 dPi(:,11) = -landv;             % dPi / dgamma2
42
43 % Manually build d2Pi (M x 11 x 11) with elements defined as in
    Equation 16
44 d2Pi = zeros(M,11,11);         % Start with Mx11x11 matrix of 0s
45
46 % lambda1 (index 1) x [alpha1, beta1-4] (indices 5-9)
47 d2Pi(:,1,5) = opop;             d2Pi(:,5,1) = opop;
48 d2Pi(:,1,6) = opop.*eld;        d2Pi(:,6,1) = opop.*eld;
49 d2Pi(:,1,7) = opop.*pinc;       d2Pi(:,7,1) = opop.*pinc;

```

```

50 d2Pi(:,1,8) = opop.*lnhdd;      d2Pi(:,8,1) = opop.*lnhdd;
51 d2Pi(:,1,9) = opop.*ffrac;      d2Pi(:,9,1) = opop.*ffrac;
52
53 % lambda2 (index 2) x [alpha1, beta1-4] (indices same)
54 d2Pi(:,2,5) = ngrw;              d2Pi(:,5,2) = ngrw;
55 d2Pi(:,2,6) = ngrw.*eld;         d2Pi(:,6,2) = ngrw.*eld;
56 d2Pi(:,2,7) = ngrw.*pinc;        d2Pi(:,7,2) = ngrw.*pinc;
57 d2Pi(:,2,8) = ngrw.*lnhdd;       d2Pi(:,8,2) = ngrw.*lnhdd;
58 d2Pi(:,2,9) = ngrw.*ffrac;       d2Pi(:,9,2) = ngrw.*ffrac;
59
60 % lambda3 (index 3) x [alpha1, beta1-4] (indices same)
61 d2Pi(:,3,5) = pgrw;              d2Pi(:,5,3) = pgrw;
62 d2Pi(:,3,6) = pgrw.*eld;         d2Pi(:,6,3) = pgrw.*eld;
63 d2Pi(:,3,7) = pgrw.*pinc;        d2Pi(:,7,3) = pgrw.*pinc;
64 d2Pi(:,3,8) = pgrw.*lnhdd;       d2Pi(:,8,3) = pgrw.*lnhdd;
65 d2Pi(:,3,9) = pgrw.*ffrac;       d2Pi(:,9,3) = pgrw.*ffrac;
66
67 % lambda4 (index 4) x [alpha1, beta1-4] (indices same)
68 d2Pi(:,4,5) = octy;              d2Pi(:,5,4) = octy;
69 d2Pi(:,4,6) = octy.*eld;         d2Pi(:,6,4) = octy.*eld;
70 d2Pi(:,4,7) = octy.*pinc;        d2Pi(:,7,4) = octy.*pinc;
71 d2Pi(:,4,8) = octy.*lnhdd;       d2Pi(:,8,4) = octy.*lnhdd;
72 d2Pi(:,4,9) = octy.*ffrac;       d2Pi(:,9,4) = octy.*ffrac;
73
74 % Compute Gradient and Hessian as in Problem Set 7
75 % Gradient:
76 grad = zeros(param,1); % Generate vector of zeros
77
78 for k=1:param
79     % Compute the "param" elements of the gradient one by one at
      each iteration of the loop
80     grad(k,1) = -sum(B.*dPi(:,k));
81 end
82
83 % Hessian:
84 hess = zeros(param); % Generate 11x11 matrix of zeros
85 for j=1:param
86
87     for k=1:param
88         % Compute each element of the hessian matrix one by one at
          each iteration of the loop

```

```

89         hess(j,k) = -sum(A.*dPi(:,j).*dPi(:,k)) - sum(B.*d2Pi(:,j,
          k));
90     end
91
92 end
93
94 end

```

A.1.5 Question C – Standard Errors and Results

```

1 % Calculate standard errors as in Equation 22
2 se      = sqrt(diag(inv(hessian)));
3
4 % Calculate Z statistic as in Equation 23
5 z        = thetahat ./ se;
6
7 % Compute p-values
8 pval     = 2*(1 - normcdf(abs(z)));
9
10 % Display results in a table
11 results = [thetahat se z pval];
12 disp('Parameter    Std Err    Z-stat    P-value');
13 disp(results);

```

A.1.6 Question E – Simulation

```

1 % Re-load and prepare data
2 data = readtable('BRdata.csv');
3 data = data(data.tire <= 4, :);
4
5 % Extract key variables
6 tpop  = data.tpop;
7 opop  = data.opop;
8 ngrw  = data.ngrw;
9 pgrw  = data.pgrw;
10 octy  = data.octy;
11 eld   = data.eld;
12 pinc  = data.pinc;
13 lnhdd = data.lnhdd;
14 ffrac = data.ffrac;

```

```

15 landv = data.landv;
16 N_obs = data.tire;
17
18 % Compute M and set N=4
19 M = length(N_obs);
20 N = 4;
21
22 %%
23 % Load coefficients from PS8
24 stata_estimators = readtable('BR_estimates.csv');
25 theta_stata = stata_estimators.coef1;
26
27 % Extract and define parameters
28 lambda = theta_stata(1:4);
29 beta = theta_stata(5:8);
30 alpha1 = theta_stata(9);
31 gammaL = theta_stata(10);
32 gamma1 = theta_stata(11);
33 alpha2 = theta_stata(12);
34 alpha3 = theta_stata(13);
35 alpha4 = theta_stata(14); % = 0 by constraint
36 alpha5 = theta_stata(15);
37 gamma2 = theta_stata(16);
38 gamma3 = theta_stata(17);
39 gamma4 = theta_stata(18);
40
41 %%
42 % Cumulative alpha and gamma for n = 1,...,4
43 alpha_n = [alpha1; alpha1-alpha2; alpha1-alpha3; alpha1-alpha4];
44 gamma_n = [gamma1; gamma1+gamma2; gamma1+gamma3; gamma1+gamma4];
45
46 % Compute S as in Equation 3
47 S = tpop + lambda(1).*opop + lambda(2).*ngrw + lambda(3).*pgrw +
    lambda(4).*octy;
48
49 % Define Pi bar as in Equation 28
50 Pi_m = zeros(M, N);
51 for n = 1:N
52     % Calculate V(n) as in Equation 26
53     Vn = alpha_n(n) + beta(1).*eld + beta(2).*pinc + beta(3)
        .*lnhdd + beta(4).*ffrac;
54     % Calculate F(n) as in Equation 27

```

```

55     Fn      = gamma_n(n) + gammaL.*landv;
56     Pi_m(:,n) = S.*Vn - Fn;
57 end
58
59 %% Simulation %%
60
61 % Step 1: Set random seed for replicability and set number of
        simulations draws to S=1000
62 rng(1);
63 S_draws = 1000;
64
65 %%
66 % Initialise vector to store expected entrants
67 E_n = zeros(M, 1);
68
69 % For each market m:
70 for m = 1:M
71     % Step 2: For each draw generate a vector of four independent
        standard normal random variables
72     eps = randn(S_draws, N);
73
74     % Initialise n*
75     n_star = zeros(S_draws, 1);
76
77     % Step 3: Loop over each random draw
78     for s = 1:S_draws
79         n_eq = 0; % start with 0 firms
80         for n = 1:N
81             % Count how many firms willing to enter given n firms
                in market
82             n_willing = sum(eps(s,:) >= -Pi_m(m,n));
83             if n_willing >= n
84                 n_eq = n; % update n*
85             else
86                 break % break the loop
87             end
88         end
89         n_star(s) = n_eq; % allocate n_eq to corresponding element
                of n_star vector
90     end
91

```

```

92     % Step 4: Average over draws to approximate  $E[n*_m \mid X_m,$ 
        theta]
93     E_n(m) = mean(n_star);
94
95 end % end market loop
96
97 %% Plot observed vs simulated entrants
98 figure;
99 scatter(E_n, N_obs, 'filled');
100 hold on;
101 plot([0 4], [0 4], 'r--', 'LineWidth', 1.5);
102 xlabel('Simulated_number_of_entrants');
103 ylabel('Observed_number_of_entrants');
104 title('Observed_vs_Simulated_Number_of_Entrants');
105 legend('Markets', '45-degree_line', 'Location', 'northwest');
106 xlim([0 4]);
107 ylim([0 4]);
108 grid on;
109 axis square;

```